

JavaScript-11

DOM-optimering

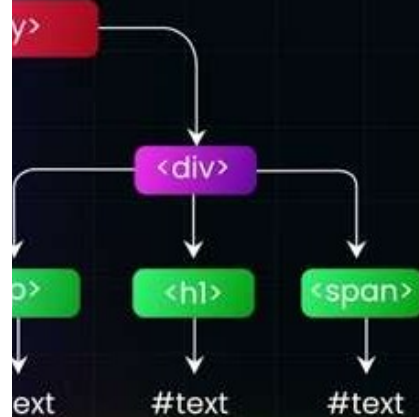
30 september 2026

JavaScript – utan tjafs

Författare: Acke Strömberg



För en kanin



DOM-optimering

När vi bygger små webbapplikationer spelar det inte så stor roll hur vi skriver vår kod. Men när applikationerna växer, och vi börjar hantera hundratals knappar eller tunga sökfält, då måste vi börja tänka på prestanda. Att manipulera och lyssna på HTML-dokumentet (DOM:en) är nämligen det "dyraste" och mest tidskrävande en webbläsare kan göra. I det här kapitlet ska vi lära oss hur vi interagerar med webbläsaren på ett smart sätt.

Event Delegation – spara på webbläsarens minne

Tänk dig att du bygger en att-göra-lista (To-Do app) eller en webbshop där du har en lista med 100 artiklar, och varje artikel har en "Ta bort"-knapp. Det vanliga sättet att lösa detta är att loopa igenom alla knappar och lägga till en händelselyssnare (`addEventListener`) på varenda en. Det betyder 100 unika lyssnare som ligger och drar minne i webbläsaren. Detta blir snabbt ineffektivt. Lösningen kallas Event Delegation. Istället för att sätta en lyssnare på varje enskild knapp, sätter vi en enda lyssnare på hela föräldra-elementet (t.ex. `<ul>`-listan).

```
[ Klicka på <button> ]  
  —→ Bubblar upp till <li>  
  —→ Bubblar upp till <ul id="minLista"> (Här fångar vi det!)
```

Bild 1: Klickhändelsen färdas uppåt som en bubbla genom alla föräldra-element.

Tack vare att händelser i JavaScript "bubblar" uppåt i HTML-trädet, kan vi fånga upp klicket på föräldern och sedan kontrollera exakt vilket element det var användaren faktiskt klickade på.

```
const lista = document.querySelector('#minLista');  
  
// Vi sätter EN ENDA lyssnare på hela listan (föräldern)  
lista.addEventListener('click', (event) => {  
  
  // event.target är det exakta elementet som muspekaren klickade på  
  if (event.target.tagName === 'BUTTON') {  
    console.log(`Klick på knappen inuti: ${event.target.parentElement.textContent}`);  
    event.target.parentElement.remove(); // Tar bort hela listobjektet (li)  
  }  
});
```

Bild 2 Genom att använda `event.target` kan vi filtrera ut klick från specifika element.

Debounce – Stoppa stressade funktioner

Vissa händelser i webbläsaren triggas otroligt ofta. Ett klassiskt exempel är när en användare skriver i ett sökfält (input) och vi vill hämta sökresultat live från ett API, eller när användaren ändrar storlek på fönstret (resize). Om en användare skriver snabbt och söker efter ordet "JavaScript" (10 bokstäver), och vi lyssnar på det vanliga input-eventet, kommer vi att skicka 10 stycken API-anrop på under två sekunder. Det stressar både webbläsaren och vår server helt i onödan. Med Debounce sätter vi en dörrvakt (en timer). Vi säger till JavaScript: "Vänta med att köra funktionen tills användaren har tagit en paus i skrivandet och det har varit tyst i 500 millisekunder".

Användaren skriver:

J — a — v — a — S — c — r — i — p — t ... (Väntar 500ms) ...
→ [API-anrop körs!]

Bild 3: Istället för ett anrop per bokstav väntar systemet tills användaren har skrivit klart.

Se kodexemplet på bild 4 för lite djupare förståelse.

```
function debounce(funktion, fördröjning) {  
  let timer;  
  
  return function(...args) {  
    clearTimeout(timer); // Avbryt den förra timern om användaren trycker på en ny tangent  
    direkt  
    timer = setTimeout(() => {  
      funktion(...args); // Kör funktionen först när tiden har gått ut helt  
    }, fördröjning);  
  };  
}  
  
// Funktionen vi vill köra mer sällan  
const hämtaSökResultat = (event) => {  
  console.log(`Hämtar data för: ${event.target.value}`);  
};  
  
const sökFält = document.querySelector('#sökInput');  
// Vi packar in vår funktion i debounce med en fördröjning på 500ms  
sökFält.addEventListener('input', debounce(hämtaSökResultat, 500));
```

Bild 4 Nu skickas endast ett samlat anrop när användaren har tagit en paus i sitt skrivande.

DocumentFragment – Bygg osynligt innan du visar

Varje gång du använder `appendChild()` för att trycka in ett nytt HTML-element på sidan måste webbläsaren räkna om hur hela designen och layouten ska ritas upp på skärmen (detta kallas för en *reflow*). Om du har en loop som lägger till 50 nya produkter i en lista efter ett API-anrop, och du kör `appendChild()` inuti loopen, tvingar du webbläsaren att göra denna tunga omräkning **50 gånger i rad**. Det kan få sidan att lagga.

```
BRA:
Loop → [ Lägg till i osynlig låda ]
      → [ Tryck ut allt på sidan samtidigt ]
(Sidan ritas om 1 gång)
-----
DÅLIGT:
Loop → [ Lägg till 1 ]
      → [ Lägg till 2 ]
      → [ Lägg till 3 ]
(Sidan ritas om 3 gånger)
```

Bild 5 Genom att samla alla element i ett fragment ritas sidan bara om en enda gång.

Lösningen är att använda ett **DocumentFragment**. Det är som en osynlig låda eller en temporär minnesyta. Du lägger till alla dina 50 element i lådan först (vilket sker blixtnabbt eftersom det sker utanför skärmen), och sedan tömmer allt från lådan på sidan **en enda gång**.

```
const produktLista = document.querySelector('#produkter');

// Vi skapar ett osynligt fragment (en skugglåda)
const fragment = document.createDocumentFragment();

const nyaProdukter = ['Skor', 'Tröja', 'Keps', 'Jacka'];

nyaProdukter.forEach((produktNamn) => {
  const li = document.createElement('li');
  li.textContent = produktNamn;

  // Vi lägger till elementet i fragmentet (inte på den riktiga skärmen än!)
  fragment.appendChild(li);
});

// Nu trycker vi ut hela paketet på skärmen på en och samma gång
produktLista.appendChild(fragment);
```

Bild 6 Webbläsaren behöver bara göra en enda layout-omräkning.

Rörelser i HTML-trädet

Man kan använda Event Delegation för att spara minne genom att lägga en enda händelselyssnare på ett föräldra-element istället för på dussintals barn. Men för att verkligen bemästra interaktiv frontend-utveckling måste vi förstå exakt hur händelser färdas genom HTML-strukturen. När en användare klickar på en knapp på din webbsida händer det nämligen mycket mer än att bara knappens funktion körs. Klicket sätter igång en hel resa genom hela ditt HTML-dokument.

Event Flow: Bubbling vs Capturing

När ett event (t.ex. ett klick) inträffar i DOM-trädet, färdas det i tre distinkta faser. Det kallas för Event Flow.

- Capturing Phase (Fångstfasen): Händelsen startar högst upp i HTML-trädet (<html>, <body>) och letar sig nedåt i strukturen tills den når det exakta elementet som användaren klickade på.
- Target Phase (Målfasen): Händelsen har nått sitt mål (t.ex. din <button>) och kör de lyssnare som sitter direkt på knappen.
- Bubbling Phase (Bubblingsfasen): Händelsen vänder om och färdas uppåt igen, från knappen hela vägen upp till <html> – precis som en luftbubbla under vatten.

Som standard lyssnar nästan alla händelselyssnare i JavaScript (addEventListener) bara i Bubblingsfasen. Det är tack vare detta som Event Delegation (från grundmaterialet) överhuvudtaget fungerar

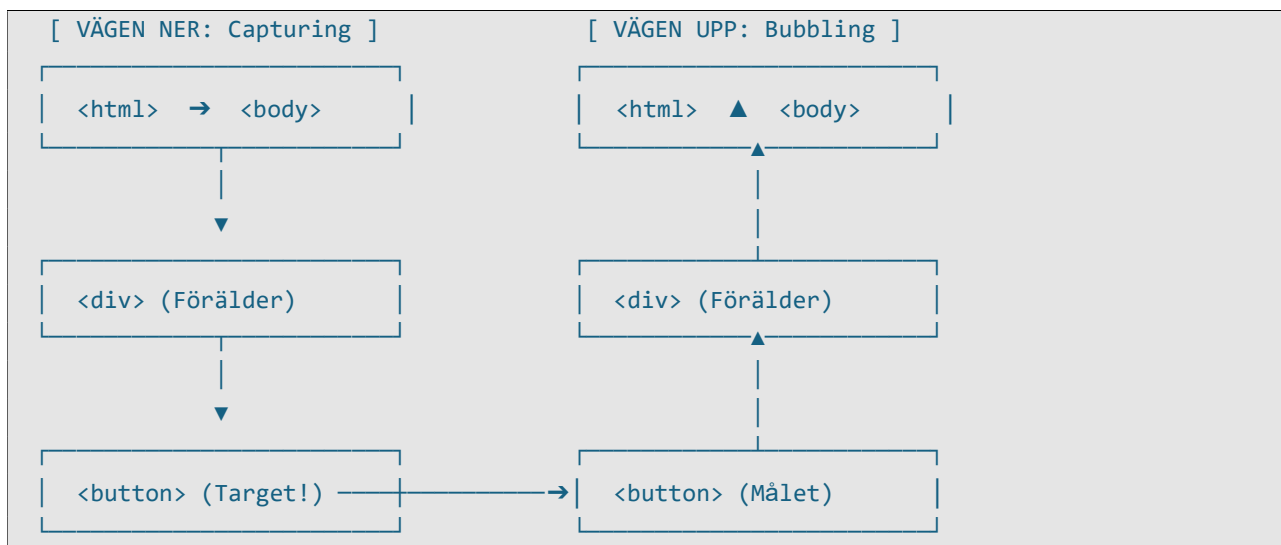


Bild 7 En händelse färdas först ner till målet (Capturing) och studsar sedan hela vägen upp till ytan (Bubbling).

stopPropagation() – Att stoppa bubblan

Ibland kan bubblingen ställa till med problem. Tänk dig att du har byggt ett pop-up-fönster (en modal). Du vill att fönstret ska stängas om användaren klickar på den gråa bakgrunden utanför fönstret, men fönstret ska självklart inte stängas om de klickar på en knapp eller skriver i ett textfält inuti fönstret. Om du har en klick-lyssnare på bakgrunden, och användaren klickar på en knapp inuti fönstret, kommer klicket att bubbla upp till bakgrunden och stänga rutan av misstag. För att förhindra detta använder vi metoden `event.stopPropagation()`. Det säger till JavaScript: "Kör den här knappens funktion, men stoppa händelsen här. Låt den inte bubbla vidare upp till mina föräldrar!".

```
const bakgrund = document.querySelector('.modal-bakgrund');
const modalFönster = document.querySelector('.modal-box');

// Stäng modalen om man klickar på bakgrunden
bakgrund.addEventListener('click', () => {
  bakgrund.style.display = 'none';
  console.log("Modalen stängdes via bakgrundsklick.");
});

// Kontrollera klick INUTI själva fönstret
modalFönster.addEventListener('click', (event) => {
  // LIVSVIKTIGT: Stoppa klicket från att bubbla upp till bakgrunden!
  event.stopPropagation();

  console.log("Du klickade inuti fönstret, modalen stängs INTE.");
});
```

Bild 8 `event.stopPropagation()` klipper av händelsens resa uppåt och isolerar klicket till det specifika elementet.

preventDefault() – Att stoppa webbläsarens natur

Webbläsaren har många inbyggda element som har ett förbestämt beteende (default behavior). Till exempel:

- När du klickar på en länk (`<a href="...">`) laddar webbläsaren automatiskt en ny sida.
- När du klickar på en Submit-knapp i ett formulär (`<form>`) laddar webbläsaren om hela sidan och skickar iväg datan i URL-en.

I modern JavaScript (och i synnerhet när vi bygger Single Page Applications med React eller Vue) vill vi sköta detta själva med vår egen kod. Vi vill till exempel att ett formulär ska skicka data i bakgrunden med Axios istället för att hela sidan ska blinka till och laddas om.

För att stänga av webbläsarens inbyggda beteende använder vi `event.preventDefault()`.

```
const formulär = document.querySelector('#mittFormular');

formulär.addEventListener('submit', (event) => {
  // Stoppa webbläsaren från att ladda om sidan!
  event.preventDefault();

  console.log("Formuläret skickades, men sidan laddades inte om.");

  // Härifrån kan vi nu köra vår egen asynkrona Axios/Fetch-kod istället!
});
```

Bild 9 `event.preventDefault()` låter utvecklaren ta full kontroll över formulär och länkar utan att webbläsarens standardbeteenden stör flödet.

Jämförelse: `preventDefault()` vs `stopPropagation()`

Det är väldigt lätt att blanda ihop dessa två metoder eftersom båda "stoppar" något. Här är den avgörande skillnaden:

- `event.preventDefault()`

Vad den gör
Stoppar webbläsarens inbyggda beteende för ett element.
Exempel på när den används
Förhindra att ett formulär laddar om sidan, eller att en länk navigerar iväg.

- `event.stopPropagation()`

Vad den gör
Stoppar händelsens resa uppåt i DOM-trädet (avbryter bubblingen).
Exempel på när den används
Förhindra att ett klick på en knapp i en lista även råkar trigga ett klick på själva listraden.